ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В. В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7

СОЕДИНЕНИЕ И ТЕСТИРОВАНИЕ КЛИЕНТСКОЙ И СЕРВЕРНОЙ ЧАСТЕЙ ВЕБ-ПРИЛОЖЕНИЯ

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		И.А. Лаврешин
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Соединить клиентские компоненты SignalBoard с серверными Route Handlers и PostgreSQL, заменить демонстрационное состояние пользователя и отслеживания реальными запросами, реализовать состояния загрузки и ошибок, проверить HTTP-контракты и авторизацию, добавить модульные и end-to-end тесты и выполнить итоговую проверку всего приложения.

2 Задание

Определить HTTP-контракт между клиентом и сервером, реализовать GET /api/signals с преобразованием Decimal-координат и вычислением isTracked для текущего пользователя. Заменить DemoAuthProvider на AuthProvider, восстанавливать сессию через GET /api/auth/me, подключить формы регистрации, входа и выхода к API, получать галерею из PostgreSQL, реализовать loading, error и empty состояния, сохранять отслеживание через POST и DELETE, обрабатывать 400, 401, 404, 409 и 500, не доверять userId и isTracked от клиента. Сохранить модальное окно, карту, меню и адаптивность. Установить Vitest и написать модульные тесты нормализации и валидации, установить Playwright и написать сквозной сценарий регистрации и отслеживания, выполнить итоговую проверку validate, migrate status, lint, unit, build и e2e.

3 Ход выполнения задания

Схема прохождения запроса: Client Component (SignalsGallery, AuthProvider) → fetch с cookie signal_session → Route Handler app/api/.../route.ts → проверка сессии (readSession, getCurrentUser) → Prisma Client → PostgreSQL → DTO в JSON → обновление состояния React.

Таблица 1 – HTTP-контракты

Метод и путь	**Тело запроса**	**Успех**	**Ошибки**
GET /api/signals	—	200 {signals: NeighborSignalDTO[]}	500
GET /api/auth/me	—	200 {user: UserDTO | null}	—
POST /api/auth/register	name, email, password, confirmPassword, gender	201 {user}	400, 409, 500
POST /api/auth/login	email, password	200 {user}	400, 401, 500
POST /api/auth/logout	—	200 {ok: true}	—
POST /api/favorites/{signalId}	—	201 {favorite}	401, 404
DELETE /api/favorites/{signalId}	—	200 {deleted}	401

UserDTO содержит id, name, email, gender и avatarUrl: passwordHash, createdAt и updatedAt клиенту не нужны, а хеш пароля нельзя раскрывать. NeighborSignalDTO содержит id, slug, title, summary, category, location, image, imageAlt, latitude, longitude, createdAt, isTracked и сведения об авторе. Координаты в базе имеют тип Decimal, который Prisma возвращает объектом Decimal.js; JSON.stringify превратил бы его в строку, поэтому сервер явно преобразует значения через Number(). Поле isTracked вычисляется на сервере по строкам TrackedSignal пользователя из проверенной сессии и не принимается от клиента. В предметной области SignalBoard маршрут GET

/api/signals соответствует шаблонному GET /api/places, а isTracked — шаблонному isFavorite.

Фрагмент GET /api/signals: выборка, Number(координат) и вычисление isTracked:

```
export function toSignalDto(
  signal: SignalRecord,
  trackedIds: Set<string>,
  currentUserId?: string | null,
) {
  return {
    id: signal.id,
    slug: signal.slug,
    title: signal.title,
    summary: signal.summary,
    category: signal.category,
    location: signal.location,
    image: signal.image,
    imageAlt: signal.imageAlt,
    latitude: Number(signal.latitude),
    longitude: Number(signal.longitude),
    createdAt: signal.createdAt.toISOString(),
    archivedAt: signal.archivedAt ? signal.archivedAt.toISOString() : null,
    isTracked: trackedIds.has(signal.id),
    isOwn: Boolean(currentUserId && signal.author.id === currentUserId),
    author: {
      id: signal.author.id,
      name: signal.author.name,
      gender: signal.author.gender,
      avatarUrl: resolveAvatarUrl(signal.author.avatarUrl,
signal.author.gender),
    },
  };
}
export async function GET() {
  try {
    const user = await getCurrentUser();

    const [signals, tracked] = await Promise.all([
      prisma.neighborSignal.findMany({
        where: { archivedAt: null },
        orderBy: { createdAt: "desc" },
        include: { author: { select: signalAuthorSelect } },
      }),
      user
        ? prisma.trackedSignal.findMany({
            where: { userId: user.id },
            select: { signalId: true },
          })
        : Promise.resolve([]),
    ]);

    const trackedIds = new Set(tracked.map((item) => item.signalId));
    return NextResponse.json({
      signals: signals.map((signal) => toSignalDto(signal, trackedIds,
user?.id)),
    });
  } catch {
    return NextResponse.json(
```

```
      { error: "Не удалось загрузить сигналы" },
      { status: 500 },
    );
  }
}
```

AuthProvider заменил DemoAuthProvider в корневом layout. При монтировании он запрашивает GET /api/auth/me и находится в состоянии loading, пока ответ не получен: без этого интерфейс на мгновение показал бы гостя вошедшему пользователю. Cookie httpOnly недоступна JavaScript, поэтому состояние восстанавливается только запросом к серверу; браузер сам прикладывает cookie к fetch с credentials: "same-origin". Функции login, register и logout вызывают соответствующие маршруты и обновляют user только после успешного ответа:

```
  const login = useCallback(async (email: string, password: string) => {
    const response = await fetch("/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "same-origin",
      body: JSON.stringify({ email, password }),
    });

    if (!response.ok) throw new Error(await readError(response));

    const payload = (await response.json()) as { user: UserDTO };
    setUser(payload.user);
    setStatus("authenticated");
  }, []);

  const register = useCallback(async (input: {
    name: string;
    email: string;
    password: string;
    confirmPassword: string;
    gender: "male" | "female";
  }) => {
    const response = await fetch("/api/auth/register", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "same-origin",
      body: JSON.stringify(input),
    });

    if (!response.ok) throw new Error(await readError(response));

    const payload = (await response.json()) as { user: UserDTO };
    setUser(payload.user);
    setStatus("authenticated");
  }, []);

  const logout = useCallback(async () => {
```

Формы входа и регистрации передают данные в AuthProvider, блокируют кнопку на время запроса и показывают сообщение сервера при ошибке:

```
  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError("");
    setPending(true);

    const form = new FormData(event.currentTarget);

    try {
      // Передаём пароль серверу только в теле HTTPS-запроса production-
среды.
      await login(String(form.get("email") ?? ""),
String(form.get("password") ?? ""));
      router.push("/gallery");
    } catch (value) {
      setError(value instanceof Error ? value.message : "Не удалось войти");
    } finally {
      setPending(false);
    }
  }
```

Загрузка галереи. SignalsGallery запрашивает /api/signals в useEffect. AbortController отменяет запрос при размонтировании или повторной загрузке, чтобы устаревший ответ не перезаписал состояние. Реализованы три состояния: loading (role="status" «Загрузка сигналов…»), error (role="alert" с кнопкой «Повторить») и empty («Сигналы пока не добавлены.»). Loading означает, что ответа ещё нет, empty – что ответ получен, но массив пуст.

```
  useEffect(() => {
    const controller = new AbortController();

    async function loadSignals() {
      setLoading(true);
      setError("");

      try {
        const response = await fetch("/api/signals", {
          cache: "no-store",
          credentials: "same-origin",
          signal: controller.signal,
        });

        if (!response.ok) throw new Error("Не удалось загрузить сигналы");

        const payload = (await response.json()) as { signals:
NeighborSignalDTO[] };
        setSignals(payload.signals);
      } catch (value) {
        if (value instanceof DOMException && value.name === "AbortError")
return;
        setError(value instanceof Error ? value.message : "Ошибка загрузки");
```

```
    } finally {
      if (!controller.signal.aborted) setLoading(false);
    }
  }

  void loadSignals();
  return () => controller.abort();
}, [reloadKey, status]);
```

Переключение отслеживания выполняется пессимистично: интерфейс меняет карточку только после того, как сервер подтвердил POST или DELETE. При ответе 401 AuthProvider обновляется и пользователь видит сообщение о завершении сеанса. userId в запрос не передаётся – сервер берёт его из подписанной сессии:

```
async function toggleTracked(signal: NeighborSignalDTO) {
  if (status !== "authenticated") return;

  setPendingId(signal.id);
  setError("");

  try {
    const response = await fetch(`/api/favorites/${signal.id}`, {
      method: signal.isTracked ? "DELETE" : "POST",
      credentials: "same-origin",
    });

    if (response.status === 401) {
      await refresh();
      throw new Error("Сеанс завершён. Выполните вход снова.");
    }

    if (!response.ok) {
      const payload = (await response.json().catch(() => null)) as
        | { error?: string; errors?: string[] }
        | null;
      const message =
        payload?.errors?.[0] ??
        payload?.error ??
        "Не удалось изменить отслеживание";
      if (response.status === 404) {
        throw new Error("Сигнал не найден");
      }
      throw new Error(message);
    }

    const nextTracked = !signal.isTracked;

    setSignals((current) =>
      current.map((item) =>
        item.id === signal.id ? { ...item, isTracked: nextTracked } : item,
      ),
    );
    setSelected((current) =>
      current && current.id === signal.id
        ? { ...current, isTracked: nextTracked }
        : current,
    );
```

```
    } catch (value) {
      setError(value instanceof Error ? value.message : "Ошибка
отслеживания");
    } finally {
      setPendingId(null);
    }
  }
```

Обработка ошибки сервера на клиенте проверена при недоступной базе данных: GET /api/signals вернул 500 без раскрытия деталей подключения (рисунок 1), а галерея показала состояние ошибки с кнопкой «Повторить» (рисунок 2).

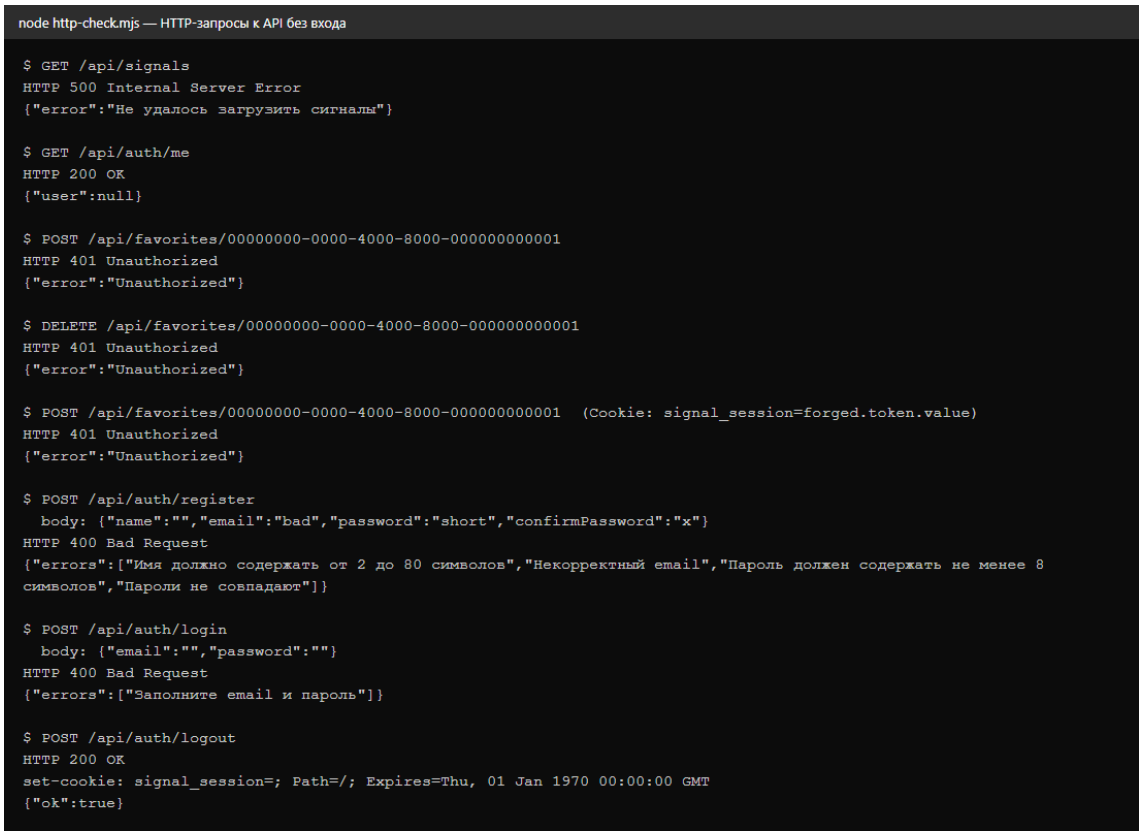


Рисунок 1 – HTTP-ответы API без входа и без базы данных

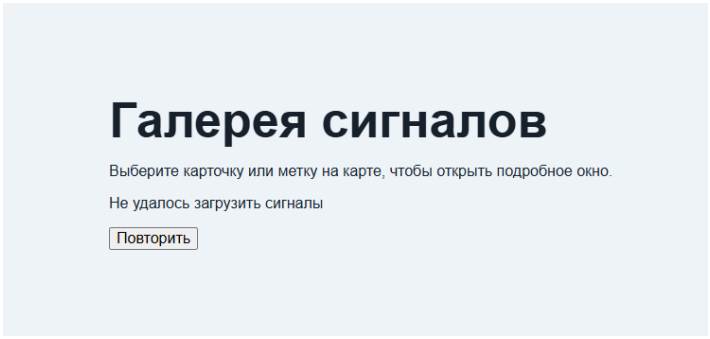


Рисунок 2 – Состояние error галереи при ответе 500

После входа seed-пользователя галерея получает данные из PostgreSQL через GET /api/signals (рисунок 3): три карточки по Плесецкой 16 и кнопки

«Отслеживать» / «В отслеживании». Состояние отслеживания сигнала «Ищу ключи от подъезда» сохраняется после перезагрузки страницы, что также видно на рисунке 3, потому что оно хранится в таблице TrackedSignal и снова читается сервером. Вкладка Network (рисунок 4) показывает GET /api/auth/me и GET /api/signals со статусом 200; в телах ответов нет passwordHash, координаты приходят числами, isTracked вычислен на сервере. SQL-проверка связи приведена на рисунке 5.

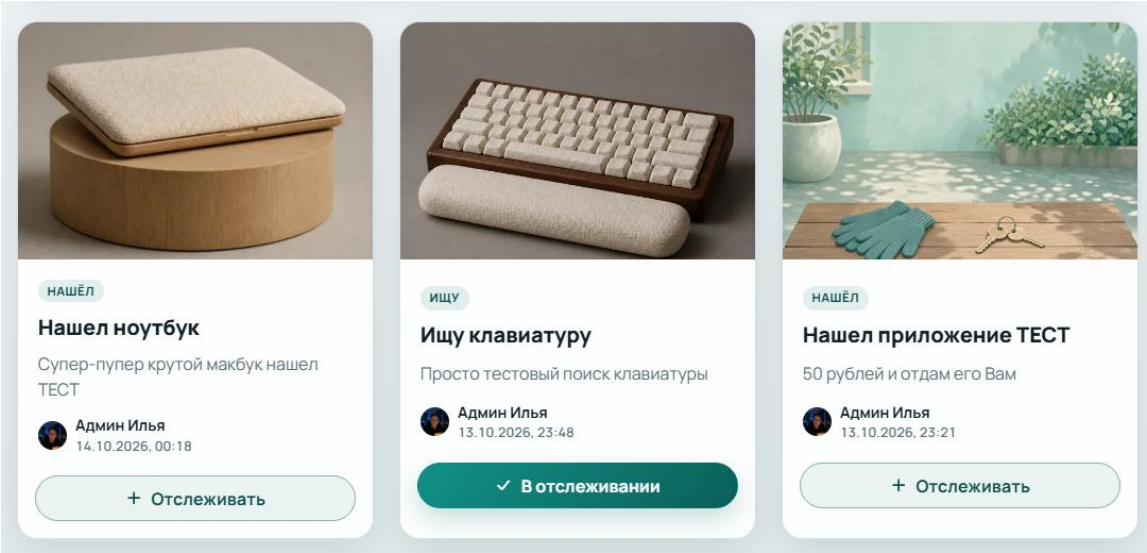


Рисунок 3 – Галерея после входа и статус отслеживания

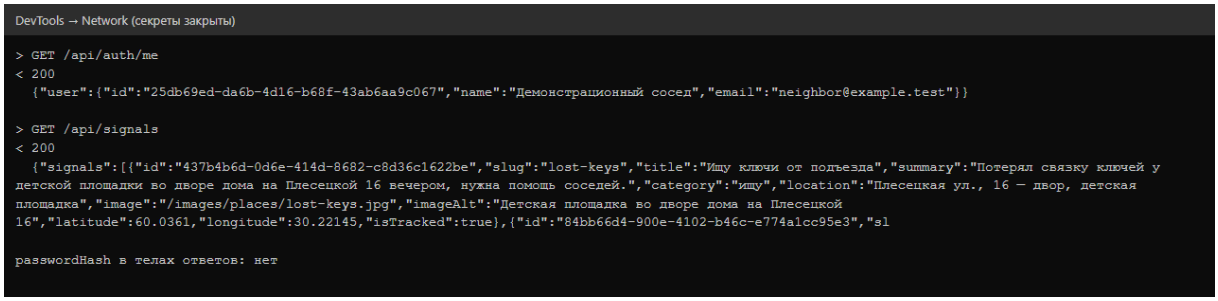


Рисунок 4 – Ответы /api/auth/me и /api/signals без passwordHash

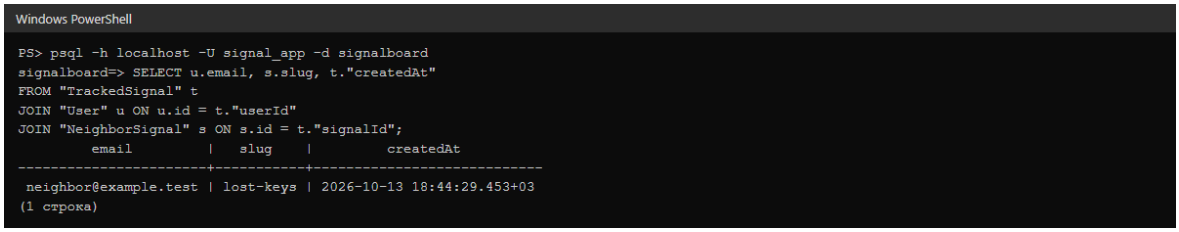


Рисунок 5 – SQL-проверка строк TrackedSignal

Полный HTTP-поток гостя, неверного пароля, входа, me, signals, POST/DELETE избранного и конфликта регистрации показан на рисунке 6. Cookie на скриншотах закрыта.

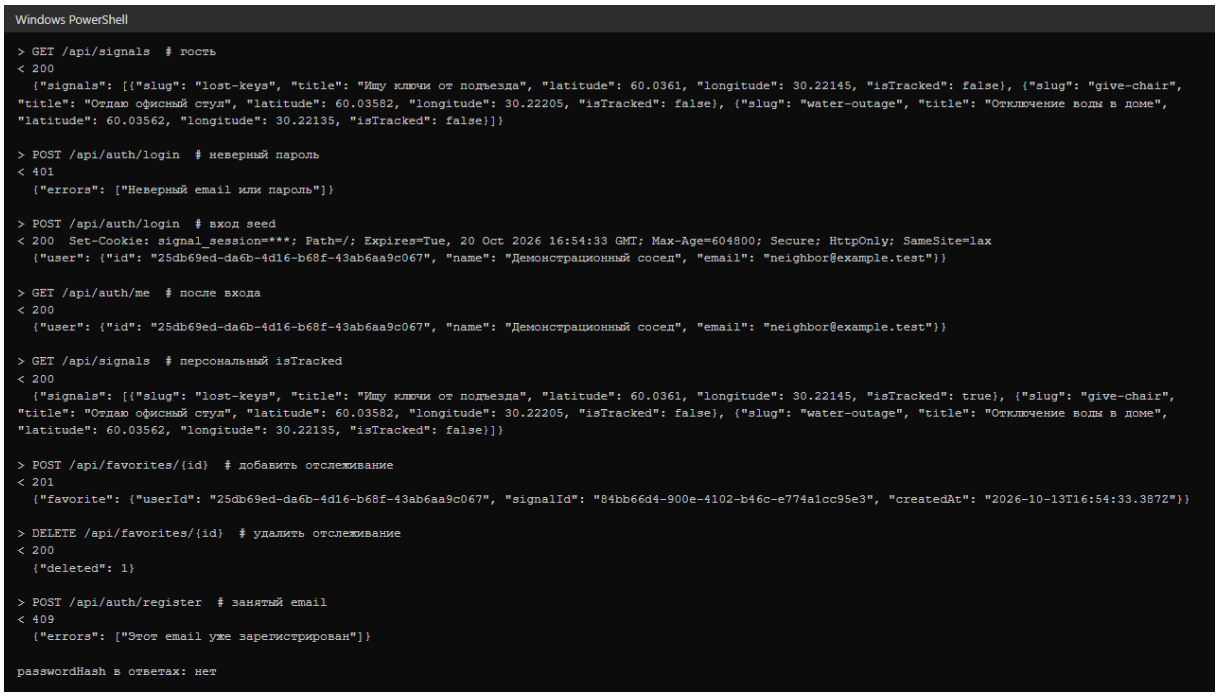


Рисунок 6 – HTTP-поток клиента с реальной базой

Таблица 2 – Матрица ручных сценариев

Сценарий	**Ожидаемый результат**	**Статус**
Гость открывает /gallery	Карточки и ссылка «Войдите, чтобы отслеживать»	Проверено
База недоступна	500 и состояние error с кнопкой «Повторить»	Проверено
Гость: POST /api/favorites/{id}	401	Проверено
Поддельная cookie	401	Проверено
Регистрация с ошибками полей	400 и сообщение в форме	Проверено
Регистрация занятого email	409 «Этот email уже зарегистрирован»	Проверено
Вход с неверным паролем	401 «Неверный email или пароль»	Проверено
Вход seed-пользователя	Переход в /gallery, кнопки отслеживания	Проверено
Отслеживание и reload	Состояние сохраняется после F5	Проверено

Сценарий	Ожидаемый результат	Статус
Выход	Cookie удалена, интерфейс гостя	Проверено (E2E и API)

Модульные тесты. Vitest настроен на окружение node и каталог tests/unit; плагин vite-tsconfig-paths подключает алиас @/. Тесты проверяют чистые функции normalizeEmail и validateRegistration и не требуют базы данных:

```
// Настраивает быстрые unit tests чистой серверной логики.

import { defineConfig } from "vitest/config";
import tsconfigPaths from "vite-tsconfig-paths";

export default defineConfig({
  // Поддерживаем алиасы путей из tsconfig.json.
  plugins: [tsconfigPaths()],
  test: {
    // Чистая валидация не требует DOM или браузера.
    environment: "node",
    include: ["tests/unit/**/*.test.ts"],
  },
});
// Проверяет чистую нормализацию и server-side validation без базы.

import { describe, expect, test } from "vitest";
import {
  normalizeEmail,
  validateRegistration,
} from "@/app/lib/validation";

describe("normalizeEmail", () => {
  test("удаляет пробелы и приводит email к нижнему регистру", () => {
    expect(normalizeEmail("  Neighbor@Example.TEST ")).toBe(
      "neighbor@example.test",
    );
  });
});

describe("validateRegistration", () => {
  test("принимает корректный набор полей", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "SignalPass_2026!",
      gender: "male",
    });

    expect(result.errors).toEqual([]);
    expect(result.data?.email).toBe("neighbor@example.test");
    expect(result.data?.gender).toBe("male");
  });

  test("отклоняет короткий пароль", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
```

```
      password: "short",
      confirmPassword: "short",
      gender: "female",
    });

    expect(result.data).toBeNull();
    expect(result.errors).toContain(
      "Пароль должен содержать не менее 8 символов",
    );
  });

  test("отклоняет несовпадающее подтверждение", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "Different_2026!",
      gender: "male",
    });

    expect(result.errors).toContain("Пароли не совпадают");
  });

  test("отклоняет регистрацию без пола", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "SignalPass_2026!",
    });

    expect(result.data).toBeNull();
    expect(result.errors).toContain("Укажите пол: мужской или женский");
  });
});
```

```
PS> npm run test:unit

> signalboard@0.1.0 test:unit
> vitest run

 RUN  v3.2.4 D:/ycheba_guap/7 семестр/Web-программирование/lab7/signalboard

 ✓ tests/unit/validation.test.ts (5 tests) 4ms

 Test Files  1 passed (1)
      Tests  5 passed (5)
   Start at  16:56:07
   Duration  1.15s (transform 44ms, setup 0ms, collect 60ms, tests 4ms, environment 0ms, prepare 651ms)
```

Рисунок 7 – Результат npm run test:unit: 5 тестов пройдены

End-to-end тест. Playwright запускает production-сервер (npm run start) через webServer и открывает страницы по baseURL. Используется один worker, потому что все тесты работают с одной локальной базой. Сценарий регистрирует пользователя с уникальным email вида e2e-<время>@example.test (с указанием пола), добавляет сигнал в отслеживание,

перезагружает страницу и проверяет, что состояние прочитано из PostgreSQL, затем выходит. В afterAll созданный пользователь удаляется, а его строки TrackedSignal удаляются каскадно, поэтому повторный запуск не накапливает тестовые данные.

```
// Запускает E2E против production-сборки SignalBoard.

import "dotenv/config";
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests/e2e",
  // Общая локальная база проверяется последовательно.
  workers: 1,
  use: {
    baseURL: "http://127.0.0.1:3000",
    trace: "on-first-retry",
    ...devices["Desktop Chrome"],
  },
  webServer: {
    // Перед test:e2e уже должна существовать npm run build.
    command: "npm run start",
    url: "http://127.0.0.1:3000",
    reuseExistingServer: !process.env.CI,
    timeout: 120_000,
  },
});
// Проверяет регистрацию, cookie-сессию и сохранение отслеживания после
reload.

import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { expect, test } from "@playwright/test";

const prisma = new PrismaClient();
let testEmail = "";

test.afterAll(async () => {
  // Удаляем только пользователя, созданного этим тестом.
  if (testEmail) {
    await prisma.user.deleteMany({ where: { email: testEmail } });
  }
  await prisma.$disconnect();
});

test("пользователь регистрируется и сохраняет сигнал в отслеживание", async
({
  page,
}) => {
  // Уникальный email исключает конфликт с предыдущим локальным запуском.
  testEmail = `e2e-${Date.now()}@example.test`;

  await page.goto("/auth/register");
  await page.getByLabel("Имя пользователя").fill("E2E сосед");
  await page.getByLabel("Email").fill(testEmail);
  // id надёжнее: «Пол» — подстрока «пользователя», getByLabel без exact
цепляет имя.
  await page.locator("#gender").selectOption("male");
```

```
  await expect(page.locator("#gender")).toHaveValue("male");
  await page.getByLabel("Пароль", { exact: true }).fill("SignalPass_2026!");
  await page.getByLabel("Повторите пароль").fill("SignalPass_2026!");
  await page.getByRole("button", { name: "Зарегистрироваться" }).click();

  // После успешного API-ответа форма переводит пользователя в галерею.
  await expect(page).toHaveURL(/\/gallery$/, { timeout: 15_000 });

  const addTracked = page.getByRole("button", {
    name: "Добавить в отслеживание: Ищу ключи от подъезда",
  });
  await addTracked.click();

  // Сервер подтвердил изменение, поэтому доступно действие удаления.
  await expect(
    page.getByRole("button", {
      name: "Удалить из отслеживания: Ищу ключи от подъезда",
    }),
  ).toBeVisible();

  // Перезагрузка повторно читает cookie и TrackedSignal из PostgreSQL.
  await page.reload();
  await expect(
    page.getByRole("button", {
      name: "Удалить из отслеживания: Ищу ключи от подъезда",
    }),
  ).toBeVisible();

  // Завершаем сессию через тот же browser context.
  await page.request.post("/api/auth/logout");
  await page.reload();
  await expect(
    page.getByRole("link", { name: "Войдите, чтобы отслеживать" }).first(),
  ).toBeVisible();
});
```

Итоговая проверка при запущенном PostgreSQL: prisma validate – схема валидна, prisma migrate status – схема актуальна (рисунок 8), npm run lint – без ошибок (рисунок 9), npm run test:unit – 5 из 5 (рисунок 10), npm run build – успешно (рисунок 11), npm run test:e2e – 1 passed (рисунок 12).

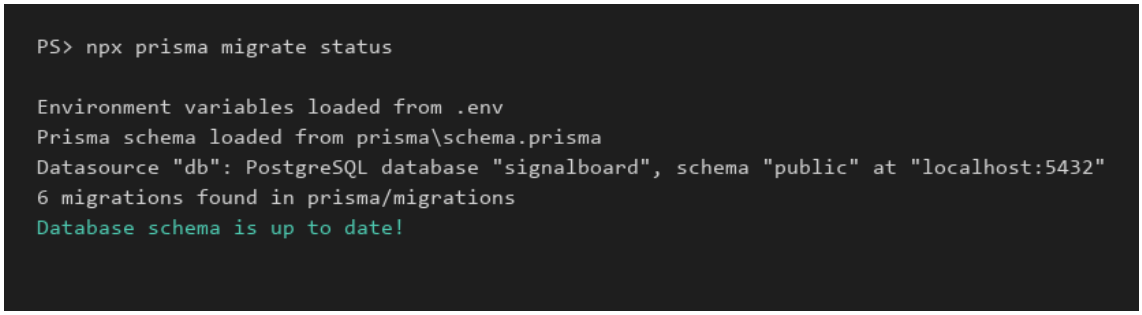


Рисунок 8 – Результат prisma migrate status

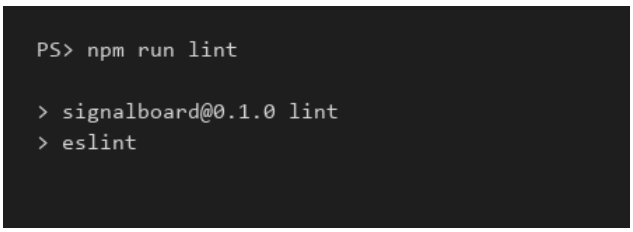


Рисунок 9 – Результат npm run lint



Рисунок 10 – Результат npm run test:unit: 5 тестов пройдены

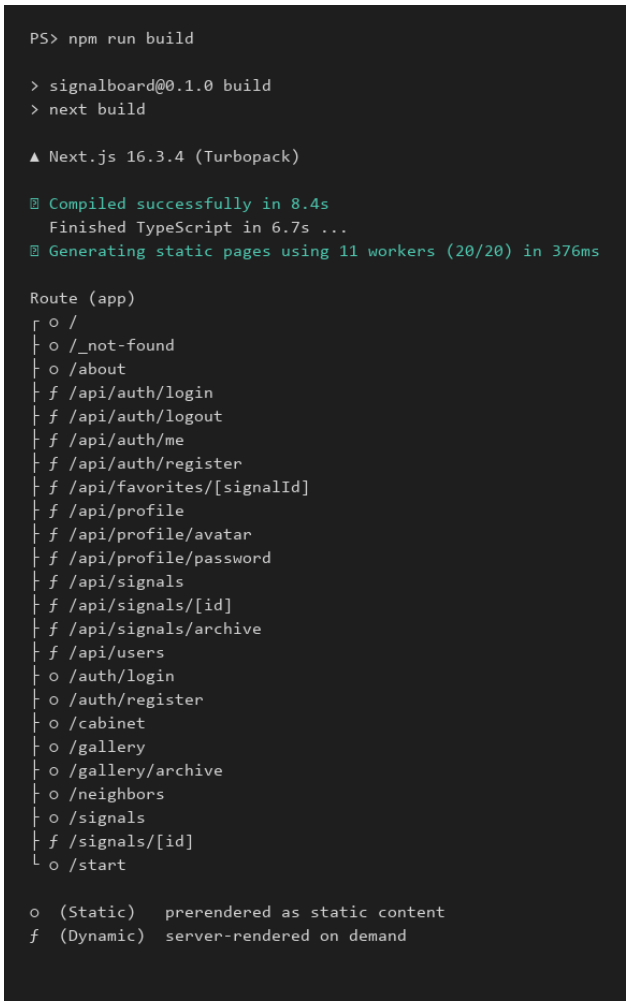


Рисунок 11 – Результат npm run build

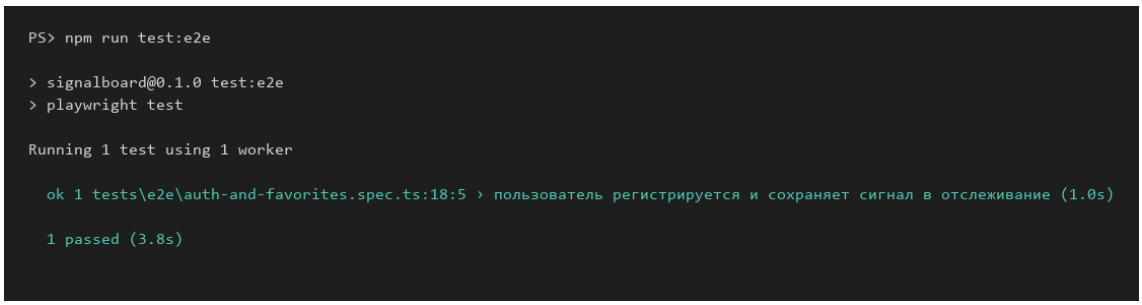


Рисунок 12 – Результат npm run test:e2e: 1 passed

Повторная ручная проверка ЛР3–ЛР4 на production-сборке ЛР7: на ширинах 320, 375, 768, 1024 и 1440 CSS px маршруты /start, /gallery, /auth/login и /auth/register не имеют горизонтальной прокрутки; бургер-меню открывается, закрывается клавишей Escape и автоматически закрывается после перехода по ссылке (рисунок 13). Модальное окно и карта используют данные из API и проверяются после запуска базы.

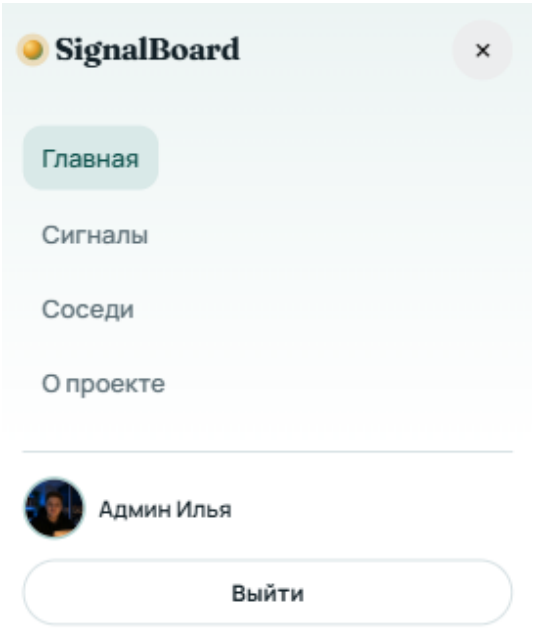


Рисунок 13 – Бургер-меню ЛР7 на ширине 375 CSS px

4 Вывод

В ходе работы клиентская часть SignalBoard соединена с серверной: галерея получает сигналы из PostgreSQL через GET /api/signals, AuthProvider восстанавливает сессию из httpOnly-cookie через /api/auth/me, формы входа и регистрации работают через API, отслеживание сохраняется POST и DELETE с серверной проверкой пользователя. Реализованы состояния loading, error и empty, отмена запросов AbortController и обработка 400, 401, 404, 409 и 500. Модульные тесты Vitest проверяют валидацию независимо от базы, сценарий Playwright проверяет регистрацию и сохранение отслеживания на production-сборке и удаляет тестовые данные. prisma migrate status подтвердил актуальность схемы, e2e-сценарий прошёл (1 passed), а состояние отслеживания после F5 совпадает с данными TrackedSignal.

Таблица 3 – Итоговая матрица проверок курса

ЛР	**Что проверено**	**Статус**
ЛР1	Каркас Next.js, маршруты, базовая вёрстка SignalBoard	Выполнено
ЛР2	Клиентская интерактивность: галерея, модальное окно, карта	Выполнено
ЛР3	Адаптивность, бургер-меню, доступность навигации	Выполнено
ЛР4	Производительность и качество (Lighthouse / ручные проверки)	Выполнено
ЛР5	PostgreSQL + Prisma: схема, миграции, seed	Выполнено
ЛР6	Route Handlers: auth, signals, favorites, сессия httpOnly	Выполнено
ЛР7	Клиент↔API, loading/error/empty, Vitest, Playwright, migrate/lint/unit/build/e2e	Выполнено

Итог по курсу. За семь лабораторных работ SignalBoard прошёл путь от каркаса Next.js до полноценного приложения: маршрутизация App Router, интерактивная клиентская часть с доступными диалогами и картой, адаптивная вёрстка с проверкой Lighthouse, реляционная база PostgreSQL с

Prisma, серверная авторизация с хешированием паролей и подписанной сессией, соединение клиента с API и автоматическое тестирование.

Курс оказался для меня действительно интересным и замотивировал на дальнейшее развитие: я планирую продолжать работу над SignalBoard, осваивать новые технологии и в перспективе довести приложение до рабочего состояния — например, чтобы использовать и продвигать его среди жителей своего дома.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

Файл app/api/signals/route.ts

```
import { randomUUID } from "crypto";
import { NextResponse } from "next/server";
import { getCurrentUser } from "@/app/lib/current-user";
import { prisma } from "@/app/lib/prisma";
import {
  defaultImageForCategory,
  isSignalCategory,
  slugifyTitle,
  validateSignalCoords,
  validateSignalFields,
} from "@/app/lib/signal-categories";
import { signalAuthorSelect, toSignalDto } from "@/app/lib/signal-dto";
import { saveSignalImage } from "@/app/lib/signal-image";

export const dynamic = "force-dynamic";
export const runtime = "nodejs";

export async function GET() {
  try {
    const user = await getCurrentUser();

    const [signals, tracked] = await Promise.all([
      prisma.neighborSignal.findMany({
        where: { archivedAt: null },
        orderBy: { createdAt: "desc" },
        include: { author: { select: signalAuthorSelect } },
      }),
      user
        ? prisma.trackedSignal.findMany({
            where: { userId: user.id },
            select: { signalId: true },
          })
        : Promise.resolve([]),
    ]);

    const trackedIds = new Set(tracked.map((item) => item.signalId));
    return NextResponse.json({
      signals: signals.map((signal) => toSignalDto(signal, trackedIds,
user?.id)),
    });
  } catch {
    return NextResponse.json(
      { error: "Не удалось загрузить сигналы" },
      { status: 500 },
    );
  }
}

export async function POST(request: Request) {
  const current = await getCurrentUser();
  if (!current) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const form = await request.formData().catch(() => null);
```

```
  if (!form) {
    return NextResponse.json({ errors: ["Некорректные данные формы"] }, {
status: 400 });
  }

  const title = String(form.get("title") ?? "").trim();
  const summary = String(form.get("summary") ?? "").trim();
  const location = String(form.get("location") ?? "").trim();
  const categoryRaw = String(form.get("category") ?? "").trim();
  const latitude = Number(form.get("latitude"));
  const longitude = Number(form.get("longitude"));
  const photo = form.get("photo");

  const errors = validateSignalFields({
    title,
    summary,
    category: categoryRaw,
    location,
  });
  const coordsError = validateSignalCoords(latitude, longitude);
  if (coordsError) errors.push(coordsError);

  if (errors.length > 0 || !isSignalCategory(categoryRaw)) {
    return NextResponse.json({ errors }, { status: 400 });
  }

  let image = defaultImageForCategory(categoryRaw).image;
  let imageAlt = defaultImageForCategory(categoryRaw).imageAlt;

  if (photo instanceof File && photo.size > 0) {
    const saved = await saveSignalImage(photo, current.id);
    if (!saved.ok) {
      return NextResponse.json({ errors: [saved.error] }, { status: 400 });
    }
    image = saved.url;
    imageAlt = saved.alt;
  }

  const slugBase = slugifyTitle(title);
  let slug = `${slugBase}-${randomUUID().slice(0, 8)}`;
  for (let attempt = 0; attempt < 3; attempt += 1) {
    const existing = await prisma.neighborSignal.findUnique({
      where: { slug },
      select: { id: true },
    });
    if (!existing) break;
    slug = `${slugBase}-${randomUUID().slice(0, 8)}`;
  }

  try {
    const signal = await prisma.neighborSignal.create({
      data: {
        slug,
        title,
        summary,
        category: categoryRaw,
        location,
        image,
        imageAlt,
        latitude: latitude.toFixed(6),
        longitude: longitude.toFixed(6),
```

```
        authorId: current.id,
      },
      include: { author: { select: signalAuthorSelect } },
    });

    return NextResponse.json(
      { signal: toSignalDto(signal, new Set(), current.id) },
      { status: 201 },
    );
  } catch {
    return NextResponse.json(
      { errors: ["Не удалось сохранить сигнал"] },
      { status: 500 },
    );
  }
}
```

Файл app/providers/AuthProvider.tsx

```
"use client";

import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useState,
  type ReactNode,
} from "react";

export type UserDTO = {
  id: string;
  name: string;
  email: string;
  gender: "male" | "female";
  avatarUrl: string | null;
};

type AuthStatus = "loading" | "authenticated" | "guest";

type AuthContextValue = {
  user: UserDTO | null;
  status: AuthStatus;
  refresh: () => Promise<void>;
  setUser: (user: UserDTO | null) => void;
  login: (email: string, password: string) => Promise<void>;
  register: (input: {
    name: string;
    email: string;
    password: string;
    confirmPassword: string;
    gender: "male" | "female";
  }) => Promise<void>;
  logout: () => Promise<void>;
};

const AuthContext = createContext<AuthContextValue | null>(null);

async function readError(response: Response): Promise<string> {
  const payload = (await response.json().catch(() => null)) as
    | { error?: string; errors?: string[] }
```

```
    | null;

  return payload?.errors?.[0] ?? payload?.error ?? "Ошибка запроса";
}

export default function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<UserDTO | null>(null);
  const [status, setStatus] = useState<AuthStatus>("loading");

  const fetchCurrentUser = useCallback(async (): Promise<UserDTO | null> => {
    const response = await fetch("/api/auth/me", {
      cache: "no-store",
      credentials: "same-origin",
    });

    if (!response.ok) {
      return null;
    }

    const payload = (await response.json()) as { user: UserDTO | null };
    return payload.user;
  }, []);

  const refresh = useCallback(async () => {
    const currentUser = await fetchCurrentUser();
    setUser(currentUser);
    setStatus(currentUser ? "authenticated" : "guest");
  }, [fetchCurrentUser]);

  useEffect(() => {
    void fetchCurrentUser().then((currentUser) => {
      setUser(currentUser);
      setStatus(currentUser ? "authenticated" : "guest");
    });
  }, [fetchCurrentUser]);

  const login = useCallback(async (email: string, password: string) => {
    const response = await fetch("/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      credentials: "same-origin",
      body: JSON.stringify({ email, password }),
    });

    if (!response.ok) throw new Error(await readError(response));

    const payload = (await response.json()) as { user: UserDTO };
    setUser(payload.user);
    setStatus("authenticated");
  }, []);

  const register = useCallback(async (input: {
    name: string;
    email: string;
    password: string;
    confirmPassword: string;
    gender: "male" | "female";
  }) => {
    const response = await fetch("/api/auth/register", {
      method: "POST",
```

```
      headers: { "Content-Type": "application/json" },
      credentials: "same-origin",
      body: JSON.stringify(input),
    });

    if (!response.ok) throw new Error(await readError(response));

    const payload = (await response.json()) as { user: UserDTO };
    setUser(payload.user);
    setStatus("authenticated");
  }, []);

  const logout = useCallback(async () => {
    const response = await fetch("/api/auth/logout", {
      method: "POST",
      credentials: "same-origin",
    });

    if (!response.ok) throw new Error(await readError(response));
    setUser(null);
    setStatus("guest");
  }, []);

  const value = useMemo(
    () => ({ user, status, refresh, setUser, login, register, logout }),
    [user, status, refresh, login, register, logout],
  );

  return <AuthContext.Provider
value={value}>{children}</AuthContext.Provider>;
}

export function useAuth(): AuthContextValue {
  const value = useContext(AuthContext);
  if (!value) throw new Error("useAuth must be used inside AuthProvider");
  return value;
}
```

Файл app/auth/login/page.tsx

```
"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useAuth } from "@/app/providers/AuthProvider";

export default function LoginPage() {
  const router = useRouter();
  const { login } = useAuth();
  const [error, setError] = useState("");
  const [pending, setPending] = useState(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError("");
    setPending(true);

    const form = new FormData(event.currentTarget);

    try {
```

```
      await login(String(form.get("email") ?? ""),
String(form.get("password") ?? ""));
      router.push("/gallery");
    } catch (value) {
      setError(value instanceof Error ? value.message : "Не удалось войти");
    } finally {
      setPending(false);
    }
  }

  return (
    <main className="form-page">
      <h1>Вход</h1>
      <form className="auth-form" method="post" onSubmit={handleSubmit}>
        <label htmlFor="email">Email</label>
        <input
          id="email"
          name="email"
          type="email"
          autoComplete="email"
          required
        />

        <label htmlFor="password">Пароль</label>
        <input
          id="password"
          name="password"
          type="password"
          autoComplete="current-password"
          minLength={8}
          required
        />

        {error ? <p role="alert" className="form-error">{error}</p> : null}
        <button type="submit" disabled={pending}>
          {pending ? "Вход…" : "Войти"}
        </button>
      </form>
      <p>
        Нет учебной записи? <Link href="/auth/register">Регистрация</Link>
      </p>
    </main>
  );
}
```

Файл app/auth/register/page.tsx

```
"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useAuth } from "@/app/providers/AuthProvider";

export default function RegisterPage() {
  const router = useRouter();
  const { register } = useAuth();
  const [error, setError] = useState("");
  const [pending, setPending] = useState(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
```

```
    setError("");
    setPending(true);

    const form = new FormData(event.currentTarget);
    const genderRaw = String(form.get("gender") ?? "");
    const gender = genderRaw === "female" ? "female" : genderRaw === "male" ?
"male" : null;

    try {
      if (!gender) throw new Error("Укажите пол: мужской или женский");

      await register({
        name: String(form.get("name") ?? ""),
        email: String(form.get("email") ?? ""),
        password: String(form.get("password") ?? ""),
        confirmPassword: String(form.get("confirmPassword") ?? ""),
        gender,
      });
      router.push("/gallery");
    } catch (value) {
      setError(value instanceof Error ? value.message : "Не удалось
зарегистрироваться");
    } finally {
      setPending(false);
    }
  }

  return (
    <main className="form-page">
      <h1>Регистрация</h1>
      <form className="auth-form" method="post" onSubmit={handleSubmit}>
        <label htmlFor="name">Имя пользователя</label>
        <input id="name" name="name" autoComplete="name" required />

        <label htmlFor="email">Email</label>
        <input id="email" name="email" type="email" autoComplete="email"
required />

        <label htmlFor="gender">Пол</label>
        <select id="gender" name="gender" required defaultValue="">
          <option value="" disabled>
            Выберите…
          </option>
          <option value="male">Мужской</option>
          <option value="female">Женский</option>
        </select>

        <label htmlFor="password">Пароль</label>
        <input
          id="password"
          name="password"
          type="password"
          autoComplete="new-password"
          minLength={8}
          required
        />

        <label htmlFor="confirmPassword">Повторите пароль</label>
        <input
          id="confirmPassword"
          name="confirmPassword"
```

```
          type="password"
          autoComplete="new-password"
          minLength={8}
          required
        />

        {error ? <p role="alert" className="form-error">{error}</p> : null}
        <button type="submit" disabled={pending}>
          {pending ? "Регистрация…" : "Зарегистрироваться"}
        </button>
      </form>
      <p>
        Уже есть учебная запись? <Link href="/auth/login">Войти</Link>
      </p>
    </main>
  );
}
```

Файл app/gallery/_components/SignalsGallery.tsx

```
"use client";

import Image from "next/image";
import Link from "next/link";
import { useCallback, useEffect, useState } from "react";
import UserAvatar from "@/app/ui/UserAvatar";
import { useAuth } from "@/app/providers/AuthProvider";
import { formatPublishedAt } from "@/app/lib/datetime";
import CreateSignalForm from "./CreateSignalForm";
import PhotoMap from "./PhotoMap";
import PhotoModal from "./PhotoModal";
import TrackToggle from "./TrackToggle";
import trackStyles from "./TrackToggle.module.css";
import styles from "./SignalsGallery.module.css";

export type NeighborSignalDTO = {
  id: string;
  slug: string;
  title: string;
  summary: string;
  category: string;
  location: string;
  image: string;
  imageAlt: string;
  latitude: number;
  longitude: number;
  createdAt: string;
  archivedAt?: string | null;
  isTracked: boolean;
  isOwn?: boolean;
  author: {
    id: string;
    name: string;
    gender: "male" | "female";
    avatarUrl: string;
  };
};

export default function SignalsGallery() {
  const { status, refresh } = useAuth();
  const [signals, setSignals] = useState<NeighborSignalDTO[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
```

```
  const [pendingId, setPendingId] = useState<string | null>(null);
  const [reloadKey, setReloadKey] = useState(0);
  const [selected, setSelected] = useState<NeighborSignalDTO | null>(null);
  const [returnFocusTo, setReturnFocusTo] = useState<HTMLButtonElement |
null>(null);

  function openFromCard(signal: NeighborSignalDTO, button: HTMLButtonElement)
{
    setReturnFocusTo(button);
    setSelected(signal);
  }

  const openFromMap = useCallback((signal: NeighborSignalDTO) => {
    setReturnFocusTo(null);
    setSelected(signal);
  }, []);

  function closeModal() {
    setSelected(null);
  }

  const retry = useCallback(() => setReloadKey((value) => value + 1), []);

  useEffect(() => {
    const controller = new AbortController();

    async function loadSignals() {
      setLoading(true);
      setError("");

      try {
        const response = await fetch("/api/signals", {
          cache: "no-store",
          credentials: "same-origin",
          signal: controller.signal,
        });

        if (!response.ok) throw new Error("Не удалось загрузить сигналы");

        const payload = (await response.json()) as { signals:
NeighborSignalDTO[] };
        setSignals(payload.signals);
      } catch (value) {
        if (value instanceof DOMException && value.name === "AbortError")
return;
        setError(value instanceof Error ? value.message : "Ошибка загрузки");
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }

    void loadSignals();
    return () => controller.abort();
  }, [reloadKey, status]);

  async function toggleTracked(signal: NeighborSignalDTO) {
    if (status !== "authenticated") return;

    setPendingId(signal.id);
    setError("");
```

```
    try {
      const response = await fetch(`/api/favorites/${signal.id}`, {
        method: signal.isTracked ? "DELETE" : "POST",
        credentials: "same-origin",
      });

      if (response.status === 401) {
        await refresh();
        throw new Error("Сеанс завершён. Выполните вход снова.");
      }

      if (!response.ok) {
        const payload = (await response.json().catch(() => null)) as
          | { error?: string; errors?: string[] }
          | null;
        const message =
          payload?.errors?.[0] ??
          payload?.error ??
          "Не удалось изменить отслеживание";
        if (response.status === 404) {
          throw new Error("Сигнал не найден");
        }
        throw new Error(message);
      }

      const nextTracked = !signal.isTracked;

      setSignals((current) =>
        current.map((item) =>
          item.id === signal.id ? { ...item, isTracked: nextTracked } : item,
        ),
      );
      setSelected((current) =>
        current && current.id === signal.id
          ? { ...current, isTracked: nextTracked }
          : current,
      );
    } catch (value) {
      setError(value instanceof Error ? value.message : "Ошибка
отслеживания");
    } finally {
      setPendingId(null);
    }
  }

  function handleCreated(signal: NeighborSignalDTO) {
    setSignals((current) => [signal, ...current]);
    setSelected(signal);
  }

  function handleUpdated(signal: NeighborSignalDTO) {
    setSignals((current) =>
      current.map((item) => (item.id === signal.id ? signal : item)),
    );
    setSelected(signal);
  }

  function handleArchived(signalId: string) {
    setSignals((current) => current.filter((item) => item.id !== signalId));
    setSelected(null);
```

```
  }

  return (
    <>
      <CreateSignalForm onCreated={handleCreated} />

      {loading ? <p role="status">Загрузка сигналов…</p> : null}

      {error && signals.length === 0 && !loading ? (
        <div role="alert">
          <p>{error}</p>
          <button type="button" className="btn btnGhost" onClick={retry}>
            Повторить
          </button>
        </div>
      ) : null}

      {error && signals.length > 0 ? <p role="alert">{error}</p> : null}

      {!loading && signals.length === 0 && !error ? (
        <p>Сигналы пока не добавлены. Опубликуйте первый.</p>
      ) : null}

      {signals.length > 0 ? (
        <div className={styles.grid}>
          {signals.map((signal, index) => (
            <article className={styles.card} key={signal.id}>
              <button
                className={styles.cardOpen}
                type="button"
                onClick={(event) => openFromCard(signal,
event.currentTarget)}
              >
                <span className={styles.imageWrap}>
                  <Image
                    src={signal.image}
                    alt={signal.imageAlt}
                    fill
                    sizes="(max-width: 47.99rem) calc(100vw - 2rem), (max-
width: 74.99rem) 45vw, 22rem"
                    className={styles.image}
                    loading={index === 0 ? "eager" : "lazy"}
                    fetchPriority={index === 0 ? "high" : "auto"}
                  />
                </span>
                <span className={styles.cardBody}>
                  <span className={styles.badge}>{signal.category}</span>
                  <span className={styles.title}>{signal.title}</span>
                  <span className={styles.summary}>{signal.summary}</span>
                  <span className={styles.author}>
                    <UserAvatar
                      name={signal.author.name}
                      avatarUrl={signal.author.avatarUrl}
                      gender={signal.author.gender}
                      className={styles.authorAvatar}
                    />
                    <span className={styles.authorMeta}>
                      <span>{signal.author.name}</span>
                      <time dateTime={signal.createdAt}
className={styles.publishedAt}>
                        {formatPublishedAt(signal.createdAt)}
                      </time>
```

```
                    </span>
                  </span>
                </span>
              </button>

              {status === "authenticated" ? (
                <TrackToggle
                  layout="card"
                  tracked={signal.isTracked}
                  pending={pendingId === signal.id}
                  title={signal.title}
                  onClick={() => void toggleTracked(signal)}
                />
              ) : (
                <Link className={trackStyles.link} href="/auth/login">
                  Войдите, чтобы отслеживать
                </Link>
              )}
            </article>
          ))}
        </div>
      ) : null}

      <PhotoMap signals={signals} onSelect={openFromMap} />

      {selected ? (
        <PhotoModal
          signal={selected}
          pending={pendingId === selected.id}
          onToggleTracked={() => void toggleTracked(selected)}
          onClose={closeModal}
          onUpdated={handleUpdated}
          onArchived={handleArchived}
          returnFocusTo={returnFocusTo}
        />
      ) : null}
    </>
  );
}
```

Файл app/layout.tsx

```
import type { Metadata, Viewport } from "next";
import type { ReactNode } from "react";
import { Fraunces, Manrope } from "next/font/google";
import AuthProvider from "./providers/AuthProvider";
import SiteHeader from "./ui/SiteHeader/SiteHeader";
import "./globals.css";

const manrope = Manrope({
  subsets: ["latin", "latin-ext", "cyrillic"],
  variable: "--font-manrope",
  display: "swap",
});

const fraunces = Fraunces({
  subsets: ["latin", "latin-ext"],
  variable: "--font-fraunces",
  display: "swap",
});

export const metadata: Metadata = {
```

```
  title: "SignalBoard",
  description: "Доска локальных сигналов соседям",
};

export const viewport: Viewport = {
  themeColor: "#0f766e",
  colorScheme: "light",
};

export default function RootLayout({ children }: { children: ReactNode }) {
  return (
    <html lang="ru" className={`${manrope.variable} ${fraunces.variable}`}>
      <body
        style={
          {
            ["--font-sans" as string]: "var(--font-manrope), 'Segoe UI',
sans-serif",
            ["--font-display" as string]: "var(--font-fraunces), Georgia,
serif",
          } as React.CSSProperties
        }
      >
        <AuthProvider>
          <SiteHeader />
          {children}
        </AuthProvider>
      </body>
    </html>
  );
}
```

Файл vitest.config.mts

```
import { defineConfig } from "vitest/config";
import tsconfigPaths from "vite-tsconfig-paths";

export default defineConfig({
  plugins: [tsconfigPaths()],
  test: {
    environment: "node",
    include: ["tests/unit*.test.ts"],
  },
});
```

Файл tests/unit/validation.test.ts

```
import { describe, expect, test } from "vitest";
import {
  normalizeEmail,
  validateRegistration,
} from "@/app/lib/validation";

describe("normalizeEmail", () => {
  test("удаляет пробелы и приводит email к нижнему регистру", () => {
    expect(normalizeEmail("  Neighbor@Example.TEST ")).toBe(
      "neighbor@example.test",
    );
  });
});

describe("validateRegistration", () => {
  test("принимает корректный набор полей", () => {
    const result = validateRegistration({
```

```
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "SignalPass_2026!",
      gender: "male",
    });

    expect(result.errors).toEqual([]);
    expect(result.data?.email).toBe("neighbor@example.test");
    expect(result.data?.gender).toBe("male");
  });

  test("отклоняет короткий пароль", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "short",
      confirmPassword: "short",
      gender: "female",
    });

    expect(result.data).toBeNull();
    expect(result.errors).toContain(
      "Пароль должен содержать не менее 8 символов",
    );
  });

  test("отклоняет несовпадающее подтверждение", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "Different_2026!",
      gender: "male",
    });

    expect(result.errors).toContain("Пароли не совпадают");
  });

  test("отклоняет регистрацию без пола", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "SignalPass_2026!",
    });

    expect(result.data).toBeNull();
    expect(result.errors).toContain("Укажите пол: мужской или женский");
  });
});
```

Файл playwright.config.ts

```
import "dotenv/config";
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests/e2e",
  workers: 1,
  use: {
    baseURL: "http://127.0.0.1:3000",
```

```
    trace: "on-first-retry",
    ...devices["Desktop Chrome"],
  },
  webServer: {
    command: "npm run start",
    url: "http://127.0.0.1:3000",
    reuseExistingServer: !process.env.CI,
    timeout: 120_000,
  },
});
```

Файл tests/e2e/auth-and-favorites.spec.ts

```
import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { expect, test } from "@playwright/test";

const prisma = new PrismaClient();
let testEmail = "";

test.afterAll(async () => {
  if (testEmail) {
    await prisma.user.deleteMany({ where: { email: testEmail } });
  }
  await prisma.$disconnect();
});

test("пользователь регистрируется и сохраняет сигнал в отслеживание", async
({
  page,
}) => {
  testEmail = `e2e-${Date.now()}@example.test`;

  await page.goto("/auth/register");
  await page.getByLabel("Имя пользователя").fill("E2E сосед");
  await page.getByLabel("Email").fill(testEmail);
  await page.locator("#gender").selectOption("male");
  await expect(page.locator("#gender")).toHaveValue("male");
  await page.getByLabel("Пароль", { exact: true }).fill("SignalPass_2026!");
  await page.getByLabel("Повторите пароль").fill("SignalPass_2026!");
  await page.getByRole("button", { name: "Зарегистрироваться" }).click();

  await expect(page).toHaveURL(/\/gallery$/, { timeout: 15_000 });

  const addTracked = page.getByRole("button", {
    name: "Добавить в отслеживание: Ищу ключи от подъезда",
  });
  await addTracked.click();

  await expect(
    page.getByRole("button", {
      name: "Удалить из отслеживания: Ищу ключи от подъезда",
    }),
  ).toBeVisible();

  await page.reload();
  await expect(
    page.getByRole("button", {
      name: "Удалить из отслеживания: Ищу ключи от подъезда",
    }),
  ).toBeVisible();
```

```
  await page.request.post("/api/auth/logout");
  await page.reload();
  await expect(
    page.getByRole("link", { name: "Войдите, чтобы отслеживать" }).first(),
  ).toBeVisible();
});
```